

Taller de Proyecto II

Informe de Avance — Octubre de 2026

G1 — ClickAula: Sistema de votación en el aula sin conexión

Grupo de Desarrollo

Valentin Ventos — 03208/4

Joaquin Labarta — 03207/3

Mateo Mignaco — 02535/2

Facultad de Informática — Universidad Nacional de La Plata

11 de Octubre de 2026

Índice general

1.- Descripción General del Proyecto	4
1.1 Propuesta original	4
1.2 Modificaciones a partir de la evaluación de la cátedra	4
1.3 Modificaciones por decisiones de diseño	5
1.4 Objetivos vigentes	5
1.5 Requerimientos	6
2.- Materiales y Presupuesto	8
2.1 Listado de materiales	8
2.2 Estado de provisión	8
2.3 Materiales dados de baja respecto del Plan	8
3.- Esquema Gráfico del Proyecto	10
3.1 Esquema general del sistema	10
3.2 Esquema de conexiones físicas del pulsador	11
3.3 Protocolo de mensajería	12
4.- Grado de Avance, Problemas y Soluciones	13
4.1 Grado de avance por componente	13
4.2 Estado de las tareas comprometidas para octubre	13
4.3 <i>Hardware</i> probado y requerimientos de consumo	14
4.4 Documentación del código desarrollado	15
4.4.1 Código que controla el <i>hardware</i>	15
4.4.2 Código que maneja las comunicaciones	15
4.4.3 Código del sistema web	16
4.4.4 <i>Software</i> que se ejecuta en la PC	17
4.5 Problemas encontrados y soluciones	18
4.6 Planificación de las partes faltantes	19
5.- Documentación Relacionada	21
5.1 Interfaz web construida	21
5.2 Partes físicas construidas	23
5.3 Enlaces	24
6.- Bibliografía	25

Índice de figuras

Figura 1. Esquema general de ClickAula: nodos publicadores, punto de acceso y <i>broker</i> , y aplicación suscriptora.	10
Figura 2. Esquema de conexiones físicas del pulsador: tapa del gabinete, cableado de los botones al NodeMCU y alimentación.	11
Figura 3. Editor de encuestas: alta de preguntas con sus opciones y la respuesta.	21
Figura 4. Panel del coordinador durante una pregunta abierta, con el temporizador, el contador de votos y la grilla de pulsadores	21
Figura 5. Vista de proyección durante una pregunta abierta	22

Figura 6. Resultados de una pregunta en el panel del coordinador, con la distribución por opción y la tabla acumulada	22
Figura 7. Pantalla final de proyección con el podio y las preguntas de mayor y menor porcentaje de acierto	23

Índice de tablas

Tabla 1. Requerimientos funcionales del sistema	6
Tabla 2. Requerimientos no funcionales del sistema	7
Tabla 3. Requerimientos de tiempo real	7
Tabla 4. Listado de materiales, cantidades y costos	8
Tabla 5. Temas de mensajería del protocolo, versión 1	12
Tabla 6. Grado de avance por componente	13
Tabla 7. Estado de las tareas comprometidas para octubre, al 6 de octubre de 2026	13
Tabla 8. Requerimientos de consumo	14
Tabla 9. Planificación cronológica de las tareas faltantes	20

1.- Descripción General del Proyecto

Esta sección consolida la introducción y los objetivos presentados en el Plan de Proyecto, incorporando las correcciones señaladas por la cátedra y las modificaciones que surgieron de decisiones de diseño tomadas durante el desarrollo. El detalle técnico de cada cambio se desarrolla en las secciones 3 y 4.

1.1 Propuesta original

Muchas escuelas no cuentan con acceso a internet en el aula, o bien prohíben de forma explícita el uso del teléfono celular durante la clase. En ese contexto, las asociaciones y los equipos de divulgación que visitan las aulas para realizar actividades participativas se quedan sin herramientas de votación en vivo: las plataformas de votación por internet dependen de que cada participante tenga un dispositivo propio, una cuenta y una conexión estable, tres condiciones que en muchas escuelas no se cumplen.

Sin esas herramientas, la actividad pierde su componente participativo. El coordinador debe estimar la distribución de respuestas a simple vista y contar manualmente, de modo que no queda registro de los resultados ni es posible comparar el desempeño entre preguntas o entre cursos.

ClickAula propone un sistema de votación autónomo para el aula. Cada participante responde desde un pulsador físico de cuatro botones de colores, y la infraestructura de red necesaria la provee el propio equipamiento del sistema, sin depender de la conexión de la escuela. El coordinador conduce la actividad y observa los resultados en vivo desde una computadora.

1.2 Modificaciones a partir de la evaluación de la cátedra

La devolución del Plan de Proyecto fue de forma y no señaló objeciones al contenido técnico. Las correcciones indicadas se aplicaron en el presente informe: se incorporaron el índice general, el índice de figuras y el índice de tablas; se agregaron epígrafes numerados debajo de cada tabla y cada figura, con su referencia correspondiente desde el texto; se incluyó la sección de bibliografía; se unificó la jerarquía de subtítulos en un único esquema numerado; se numeraron los requerimientos para poder referenciarlos; se definieron las siglas en su primer uso; se normalizó la tipografía del cuerpo en 12 pt con interlineado 1,5; y se reemplazó el pie de página por la numeración de páginas.

Respecto del contenido, se atendieron dos observaciones puntuales. La introducción fue reescrita para presentar el problema y el contexto sin entrar en especificaciones técnicas, que ahora se desarrollan en las secciones 3 y 4, y se eliminaron las expresiones coloquiales. La figura del esquema general fue regenerada con rótulos de mayor tamaño y en color negro, de modo que resulte legible al tamaño de impresión.

1.3 Modificaciones por decisiones de diseño

Durante el desarrollo se tomaron tres decisiones que modifican lo presentado en el Plan de Proyecto.

Eliminación del LED de confirmación. El Plan preveía un diodo emisor de luz en cada pulsador que se encendía al registrarse el voto. Se decidió retirarlo y trasladar la confirmación a la pantalla de proyección, donde una grilla muestra qué pulsadores ya respondieron. La consecuencia es de alcance mayor al componente retirado: sin un elemento que encender, el pulsador deja de necesitar mensajes entrantes y se convierte en un publicador puro, lo que elimina del protocolo los temas de control y de acuse de recibo, y simplifica de manera considerable el *firmware*.

Supresión del regulador de tensión externo. El Plan incorporaba un regulador reductor dedicado para alimentar el módulo sin tomar tensión de la placa de desarrollo, siguiendo la recomendación de la cátedra. Al eliminarse el LED, el pulsador quedó sin cargas externas: los botones son pasivos y el único consumidor del circuito es el propio módulo. La recomendación, que apunta a no alimentar periféricos desde el regulador de la placa, dejó de tener objeto, por lo que el módulo se alimenta por su conector micro-USB desde una batería portátil de 5 V, que es la configuración prevista por el fabricante de la placa.

Reemplazo de la placa perforada por una *protoboard*. El Plan preveía un montaje soldado sobre placa perforada. Dado que el circuito se redujo a cuatro botones y ocho cables, se optó por una *protoboard* alojada dentro del gabinete, con los botones atornillados a la tapa. Los botones, que son la parte expuesta al uso, quedan firmemente sujetos, mientras que la *protoboard* no recibe manipulación directa.

1.4 Objetivos vigentes

Objetivo general. Construir un sistema de votación de respuesta múltiple para el aula, completamente autónomo y sin conexión a internet, en el que cada participante responde

desde un pulsador físico de cuatro opciones y el coordinador conduce la actividad y observa los resultados en vivo desde una computadora.

Objetivos particulares. Los objetivos particulares mantienen lo comprometido en el Plan, con las modificaciones de la sección 1.3:

- Construir un pulsador de cuatro botones de colores con NodeMCU (ESP8266) que publique cada pulsación.
- Configurar una Raspberry Pi 3 como punto de acceso inalámbrico autónomo y como *broker*, sin dependencia de la infraestructura de red de la escuela.
- Definir un protocolo de mensajes propio sobre MQTT (Message Queuing Telemetry Transport), con su formato de mensaje, su nivel de calidad de servicio (QoS, por Quality of Service) y su política ante desconexiones.
- Desarrollar la aplicación suscriptora que abre y cierra cada votación, persiste los eventos, evalúa las respuestas y presenta el tablero que conduce el coordinador.
- Validar el comportamiento del sistema con pulsadores concurrentes y con una prueba de carga representativa de un aula de aproximadamente treinta dispositivos.

El patrón de comunicación adoptado es publicador/suscriptor sobre MQTT, protocolo de mensajería ampliamente utilizado en aplicaciones de internet de las cosas (IoT, por Internet of Things) con nodos de recursos limitados [1].

1.5 Requerimientos

Los requerimientos se numeran para poder referenciarlos desde los informes de avance y desde las pruebas, según lo indicado en la devolución. Los requerimientos funcionales se listan en la Tabla 1, los no funcionales en la Tabla 2 y los de tiempo real en la Tabla 3.

Id.	Requerimiento
RF-1	El coordinador carga un cuestionario de preguntas con hasta cuatro opciones y, opcionalmente, una respuesta correcta.
RF-2	Al abrir una votación los pulsadores quedan habilitados para responder; al cerrarla, las pulsaciones dejan de registrarse.
RF-3	Cada participante emite un único voto por pregunta. Se conserva la primera pulsación y se descartan las repeticiones.
RF-4	El sistema registra cada voto con su identificador de pulsador, de pregunta, la opción elegida y la marca de tiempo de recepción.
RF-5	En la modalidad competitiva el sistema asigna puntaje según el orden de llegada entre las respuestas correctas y mantiene una tabla de posiciones acumulada.
RF-6	En la modalidad de encuesta anónima no se asignan puntajes ni se identifica a los participantes.
RF-7	Al finalizar la sesión el sistema almacena los resultados agregados y los expone para su consulta posterior.

Tabla 1. Requerimientos funcionales del sistema.

Id.	Requerimiento
RNF-1	El sistema opera sin conexión a internet ni a la red de la escuela. La red inalámbrica la genera el propio equipamiento.
RNF-2	El diseño soporta del orden de treinta pulsadores concurrentes.
RNF-3	El equipamiento completo se transporta y se pone en funcionamiento en pocos minutos, sin configuración por parte de la escuela.
RNF-4	Cada pulsador funciona sin conexión a la red eléctrica, con autonomía superior a una jornada de actividad.
RNF-5	La caída o el reinicio de un pulsador no interrumpe la sesión ni afecta a los demás nodos.
RNF-6	El participante interactúa únicamente con cuatro botones, sin pantalla, cuentas ni configuración.

Tabla 2. Requerimientos no funcionales del sistema.

El sistema es de tiempo real blando: el incumplimiento de un plazo degrada la experiencia de la actividad pero no produce una falla crítica. Los plazos comprometidos se listan en la Tabla 3.

Id.	Restricción temporal	Valor objetivo
RT-1	Latencia desde la pulsación hasta el registro del voto	menor a 500 ms
RT-2	Propagación de la apertura o el cierre de la votación	menor a 1 s
RT-3	Período del latido de presencia de cada pulsador	5 s
RT-4	Umbral para dar por caído a un pulsador sin latido	15 s
RT-5	Antirrebote de cada botón en el <i>firmware</i>	50 ms

Tabla 3. Requerimientos de tiempo real.

La marca de tiempo que ordena los votos es la que asigna la aplicación al recibirlos, dado que los módulos ESP8266 no disponen de reloj de tiempo real ni de acceso a un servidor de hora.

2.- Materiales y Presupuesto

2.1 Listado de materiales

El listado de la Tabla 4 corresponde al armado de dos pulsadores y del nodo central, y refleja las modificaciones indicadas en la sección 1.3. No se incluye la computadora portátil del grupo, que se utiliza como nodo suscriptor y como equipo de desarrollo.

Componente	Cant.	Costo unitario	Referencia
Raspberry Pi 3 modelo B o B+	1	\$123.689	MercadoLibre
Tarjeta microSD de 16 GB clase 10	1	\$14.609	MercadoLibre
Fuente de 5 V – 2,5 A con conector micro-USB	1	\$17.182	MercadoLibre
Gabinete para Raspberry Pi	1	\$8.998	MercadoLibre
NodeMCU con ESP8266	2	\$9.359	MercadoLibre
Pulsador momentáneo de panel de 16 mm, con color	8	\$3.327	MercadoLibre
protoboard de 400 puntos	2	\$4.640	MercadoLibre
Juego de cables de conexión	1	\$5.219	MercadoLibre
Batería portátil de 5 V, 2000 a 2500 mAh	2	\$17.000	MercadoLibre
Cable micro-USB de 20 a 30 cm	2	\$3.000	MercadoLibre
Gabinete plástico de 110 × 60 × 30 mm	2	\$11.600	MercadoLibre
Estaño para soldar	1	\$4.000	MercadoLibre
Total		\$291.511	

Tabla 4. Listado de materiales, cantidades y costos.

2.2 Estado de provisión

El grupo no dispone de ninguno de los componentes listados. La totalidad de los materiales fue solicitada a la cátedra mediante el pedido enviado por los canales institucionales, y su entrega se encuentra pendiente a la fecha de este informe. En consecuencia, ningún componente fue adquirido con fondos propios ni con fondos provistos por la cátedra hasta el momento.

Esta situación condiciona el avance de las tareas de *hardware*, según se detalla en las secciones 4.2 y 4.6. El desarrollo de la aplicación, en cambio, no se vio afectado, dado que se previó desde el inicio un entorno de simulación que reemplaza al equipamiento físico.

2.3 Materiales dados de baja respecto del Plan

Respecto del listado presentado en el Plan de Proyecto se dieron de baja los siguientes componentes, como consecuencia de las decisiones de la sección 1.3: los diodos emisores de luz y sus resistencias limitadoras de 220 Ω ; los módulos reguladores reductores de 5 V a

3,3 V; las resistencias de 10 k Ω , reemplazadas por las resistencias de *pull-up* del ESP8266, que se habilitan por *software*; y las placas perforadas, reemplazadas por placas de pruebas.

3.- Esquema Gráfico del Proyecto

3.1 Esquema general del sistema

El sistema se organiza en tres partes físicas, como muestra la Figura 1.

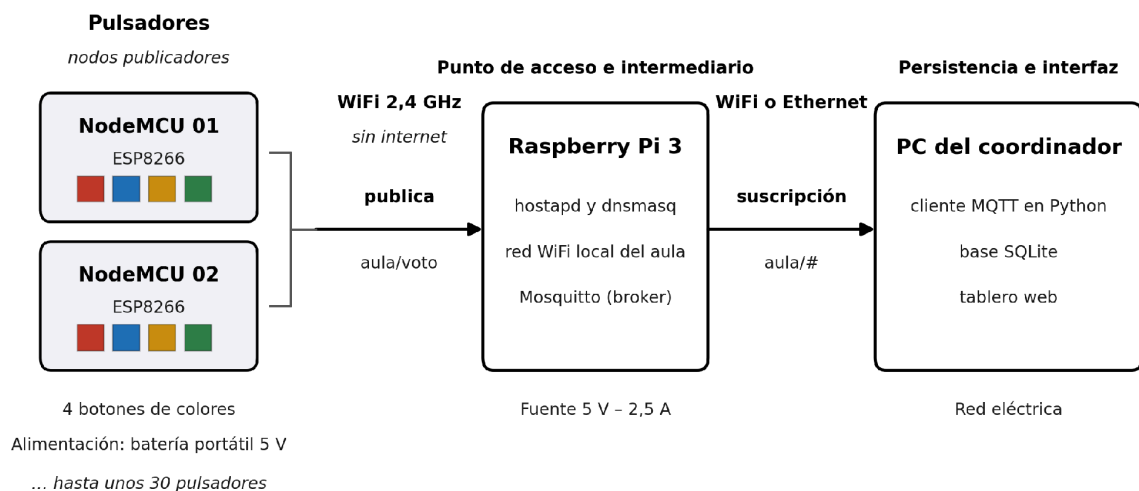


Figura 1. Esquema general de ClickAula: nodos publicadores, punto de acceso y broker, y aplicación suscriptora.

Pulsadores. Cada uno está construido sobre un NodeMCU con ESP8266 y cuenta con cuatro botones de colores y su propia batería. Se asocian a la red inalámbrica de la Raspberry Pi y publican cada pulsación. No reciben mensajes del sistema.

Raspberry Pi 3. Genera la red inalámbrica local del aula mediante los servicios hostapd y dnsmasq, y ejecuta el *broker* Mosquitto, que encamina los mensajes entre publicadores y suscriptores. No procesa ni almacena votos.

Computadora del coordinador. Se suscribe a los temas del aula, persiste cada evento en la base de datos, evalúa las respuestas y sirve el tablero web. Es también quien determina la apertura y el cierre de cada votación.

El flujo de una pregunta es el siguiente: el coordinador abre la votación desde el tablero y la aplicación inicia la ventana de recepción; cada participante pulsa y su nodo publica el voto; la aplicación lo recibe, valida que corresponda a la pregunta en curso y lo registra; al vencer el temporizador la ventana se cierra y los votos posteriores se descartan; finalmente

el tablero muestra la distribución de respuestas. En paralelo, cada pulsador emite un latido periódico de presencia que permite conocer qué dispositivos se encuentran activos.

3.2 Esquema de conexiones físicas del pulsador

La Figura 2 presenta el esquema de conexiones físicas entre la placa de desarrollo y los dispositivos, tal como se solicita en la plantilla de este informe.

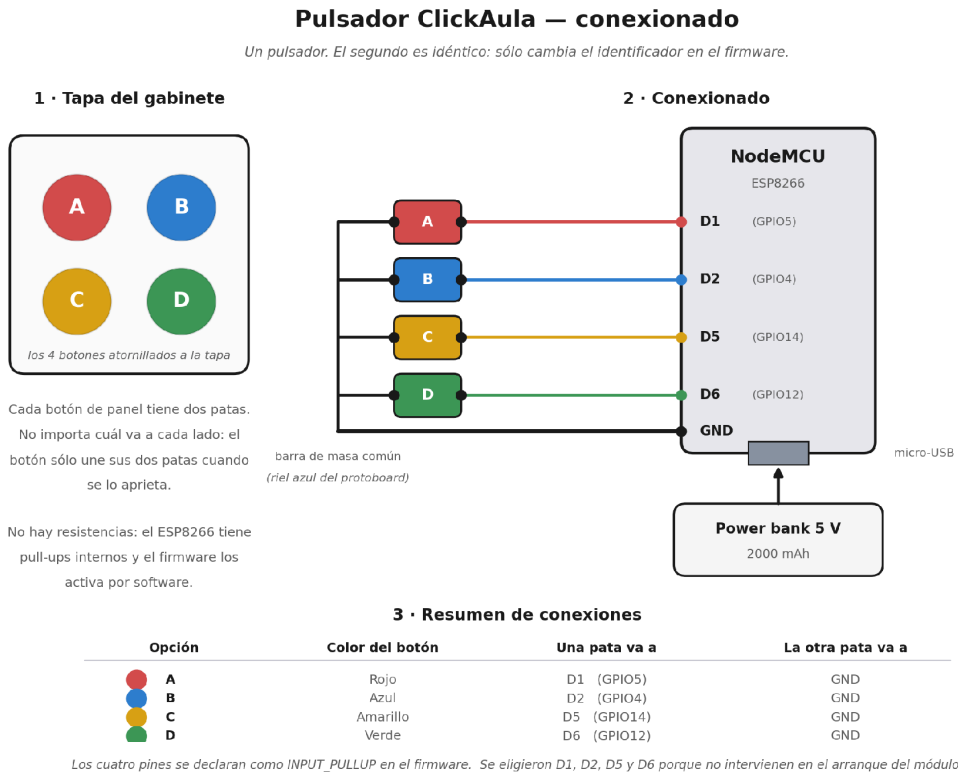


Figura 2. Esquema de conexiones físicas del pulsador: tapa del gabinete, cableado de los botones al NodeMCU y alimentación.

Cada botón conecta una de sus patas al pin indicado del NodeMCU y la otra a masa. Las cuatro patas de masa se unen en un riel común de la *protoboard*, del que sale un único conductor hacia el pin de masa del módulo. La alimentación llega por el conector micro-USB desde la batería portátil.

Los pines se eligieron de modo de no interferir con el arranque del módulo. El ESP8266 consulta el estado de determinadas entradas en el momento del encendido para seleccionar su modo de operación, por lo que un botón conectado a una de ellas y presionado durante el encendido impediría la ejecución normal del programa. Los pines D1, D2, D5 y D6 no intervienen en esa secuencia y admiten resistencia de elevación interna.

3.3 Protocolo de mensajería

El protocolo quedó definido en su versión 1 y constituye la interfaz entre el *firmware* y la aplicación. Los temas se listan en la Tabla 5. Como consecuencia de la eliminación del LED, el pulsador no se suscribe a ningún tema y la aplicación no publica mensaje alguno dirigido a él.

Tema	Publica	Contenido y calidad de servicio
aula/voto/<id>	Pulsador	Opción elegida y número de secuencia propio del nodo. QoS 1, con deduplicación por secuencia en la aplicación.
aula/presencia/<id>	Pulsador	Latido periódico cada 5 s. QoS 0, retenido, con mensaje de última voluntad que marca al nodo como caído si se pierde la conexión.

Tabla 5. Temas de mensajería del protocolo, versión 1.

El mensaje de voto no incluye el número de pregunta ni una marca de tiempo: la aplicación asigna el voto a la pregunta que se encuentra abierta en el instante de recepción y le aplica su propio reloj. Esta decisión reduce el estado que debe mantener el pulsador y elimina la necesidad de sincronizar relojes entre nodos.

4.- Grado de Avance, Problemas y Soluciones

4.1 Grado de avance por componente

La Tabla 6 resume el grado de avance de cada componente del proyecto a la fecha de este informe.

Componente	Avance	Observaciones
Protocolo de mensajería	100 %	Versión 1 cerrada y documentada. Queda abierta una decisión sobre la calidad de servicio, descrita en la sección 4.5.
Aplicación de votación	90 %	Funcionando en sus dos modalidades, con persistencia y tablero. Resta corregir el problema de deduplicación de la sección 4.5.
Simulador de pulsadores	100 %	Reemplaza al <i>hardware</i> para desarrollo y pruebas.
Entorno de desarrollo	100 %	<i>broker</i> , aplicación y simulador se levantan con un único comando.
<i>Firmware</i> del pulsador	0 %	Sin iniciar. Comienza con la recepción de los materiales; el diseño y el esqueleto de referencia están definidos.
Raspberry Pi como punto de acceso	20 %	Investigación concluida, con los obstáculos identificados. La implementación comienza con la recepción del equipamiento.
Armado físico de los pulsadores	0 %	Sin iniciar. Esquema de conexiones ya definido (Figura 2).

Tabla 6. Grado de avance por componente.

4.2 Estado de las tareas comprometidas para octubre

El Plan de Proyecto comprometió siete tareas con plazo al cierre de octubre. Dado que el presente informe se redacta al comienzo del mes, ese plazo se encuentra en curso: la Tabla

7 refleja el estado de avance a la fecha y no una evaluación de cumplimiento final, que corresponderá al cierre del período.

Tarea comprometida	Estado	Observaciones
Protocolo de mensajería definido y documentado	Cumplida	Se simplificó respecto de lo previsto al eliminarse el LED.
Aplicación de votación terminada	Cumplida	Incluye las dos modalidades, la persistencia y el tablero.
Simulador de pulsadores funcionando	Cumplida	Parametrizable en cantidad, acierto, demora y participación.
Raspberry Pi configurada como punto de acceso	En plazo	Investigación concluida. La implementación comienza con la recepción de los materiales.
<i>Firmware</i> base del pulsador	En plazo	Diseño y esqueleto de referencia definidos. Comienza con la recepción de los materiales.
Primer pulsador armado sobre <i>protoboard</i>	En plazo	Esquema de conexiones definido. Comienza con la recepción de los materiales.
Demostración de extremo a extremo con un pulsador físico	En plazo	Ya realizada con pulsadores simulados sobre el protocolo definitivo. Resta repetirla sobre equipamiento real.

Tabla 7. Estado de las tareas comprometidas para octubre, al 6 de octubre de 2026.

Tres de las siete tareas se encuentran concluidas y las cuatro restantes están en plazo. Las tres tareas concluidas son las de *software*, que el Plan identificaba como independientes del *hardware* y que se completaron de manera anticipada respecto del cierre del mes. Entre ellas se encuentra la aplicación de votación, que constituía el entregable principal del período.

Las cuatro tareas restantes dependen de la recepción de los materiales, pendiente a la fecha según se indica en la sección 2.2. Esta dependencia estaba prevista en el Plan de Proyecto, que señalaba que las tareas de configuración y armado quedaban condicionadas a la disponibilidad del equipamiento mientras que el desarrollo de la aplicación no lo estaba. La previsión resultó acertada: el desarrollo avanzó contra el simulador y un *broker* local sin verse afectado por la demora.

4.3 Hardware probado y requerimientos de consumo

Hardware probado. A la fecha de este informe no se ha probado ningún componente físico, por encontrarse pendiente la recepción de los materiales. La validación se realizó íntegramente sobre el entorno de simulación descrito en la sección 4.4.4, con clientes que reproducen el comportamiento de red previsto para los pulsadores: publicación de presencia retenida, latido periódico, mensaje de última voluntad y publicación de votos con número de secuencia.

Requerimientos de consumo. La Tabla 8 detalla los consumos estimados de cada parte y del sistema completo, recalculados a partir de las modificaciones de la sección 1.3. Al eliminarse el LED y el regulador externo, el único consumidor del pulsador es el módulo, alimentado a 5 V por su conector micro-USB. Los valores corresponden a la hoja de datos del ESP8266 [2] y a la recomendación del fabricante de la Raspberry Pi [3]; deberán contrastarse con mediciones una vez que se disponga del equipamiento.

Parte	Tensión	Corriente media	Corriente de pico
NodeMCU con ESP8266 (por pulsador)	5 V	80 mA	200 mA en transmisión
Botones con elevación interna	3,3 V	despreciable	despreciable
Raspberry Pi 3 en modo punto de acceso	5 V	400 a 700 mA	1,2 A
Sistema completo con dos pulsadores	5 V	0,56 a 0,86 A	1,6 A

Tabla 8. *Requerimientos de consumo por parte y del sistema completo.*

Con un consumo medio de 80 mA, una batería portátil de 2000 mAh supera las quince horas de autonomía por pulsador, holgadamente por encima de lo exigido por el requerimiento RNF-4. Resta verificar que el modelo provisto no incorpore apagado automático por baja demanda, comportamiento habitual en estos dispositivos por debajo de los 50 a 70 mA.

4.4 Documentación del código desarrollado

El código desarrollado se encuentra en el repositorio indicado en la sección 5.3. Las dependencias externas se limitan a dos bibliotecas de Python: Flask [4] para el servidor web y paho-mqtt [5] para el cliente de mensajería. La interfaz web no utiliza ninguna biblioteca externa, decisión derivada del requerimiento RNF-1: una referencia a un servidor de distribución de contenidos impediría el funcionamiento en un aula sin conexión.

4.4.1 Código que controla el *hardware*

El *firmware* que controla los botones del pulsador no ha sido desarrollado, por las razones indicadas en la sección 4.2. Su diseño quedó especificado y se dispone de un esqueleto de referencia, cuyo núcleo es la lectura de los cuatro botones con antirrebote y la publicación del voto:

```
for (int i = 0; i < 4; i++) {
    bool nivel = digitalRead(PINES[i]);          // LOW = apretado
    if (nivel != estadoPrevio[i] && ahora - ultimoCambio[i] > 50) {
        ultimoCambio[i] = ahora;
        estadoPrevio[i] = nivel;
        if (nivel == LOW) {                      // flanco de pulsación
            char payload[40];
```

```

    snprintf(payload, sizeof(payload),
             "{\"opcion\":\"%s\",\"seq\":%lu}", OPCION[i], ++seq);
    mqtt.publish(topicVoto, payload, false);
  }
}
}

```

Las entradas se declaran con resistencia de elevación interna, por lo que la lógica queda invertida: el pin se lee en nivel alto en reposo y en nivel bajo al presionarse el botón. La comparación con el estado previo asegura que se publique un único voto por pulsación y no de forma continua mientras el botón permanece presionado, y el umbral de 50 ms implementa el antirrebote exigido por el requerimiento RT-5.

4.4.2 Código que maneja las comunicaciones

La comunicación con los pulsadores es la única entrada de datos del sistema y está concentrada en un módulo. La función que recibe cada mensaje del intermediario es la siguiente:

```

def _on_message(client, userdata, msg):
    pid = _pulsador_de(msg.topic)
    try:
        payload = json.loads(msg.payload.decode("utf-8") or "{}")
    except (ValueError, UnicodeDecodeError):
        print(f"[mqtt] payload ilegible en {msg.topic}", flush=True)
        return

    if msg.topic.startswith("aula/voto/"):
        opcion = str(payload.get("opcion", "")).upper()
        motivo = motor.votar(pid, opcion, payload.get("seq"))
        if motivo:
            print(f"[voto] {pid} {opcion} descartado: {motivo}", flush=True)

    elif msg.topic.startswith("aula/presencia/"):
        online = bool(payload.get("online", False))
        motor.visto(pid, online=online, retenido=bool(msg.retain))

```

El módulo no toma decisiones: identifica el pulsador a partir del último segmento del tema, interpreta el contenido del mensaje y delega en el motor de la partida. Un mensaje con formato inválido se descarta sin interrumpir la recepción. El indicador de mensaje retenido permite distinguir un latido real de un mensaje que el intermediario conservó de una sesión anterior, de modo que los pulsadores de sesiones previas no se contabilicen como presentes.

4.4.3 Código del sistema web

La interfaz web consta de siete plantillas HTML, cinco archivos de JavaScript y una hoja de estilos, y se sirve desde la computadora del coordinador. Se compone de dos vistas: el panel del coordinador, donde se cargan las encuestas y se conduce la actividad, y la vista

de proyección, destinada a la pantalla del aula. Las capturas correspondientes se presentan en la sección 5.1.

La actualización en vivo se resuelve con eventos enviados por el servidor (SSE, por Server-Sent Events), que es un mecanismo del propio protocolo HTTP y no requiere bibliotecas adicionales. El servidor envía el estado completo en cada cambio y el navegador se limita a dibujarlo:

```
function escucharEstado(alRecibir) {
  let fuente = null;

  function conectar() {
    fuente = new EventSource("/api/stream");
    fuente.onmessage = (ev) => alRecibir(JSON.parse(ev.data));
    fuente.onerror = () => { // reconexión automática
      fuente.close();
      setTimeout(conectar, 1500);
    };
  }

  conectar();
}
```

Enviar el estado completo en lugar de cambios incrementales evita los errores de desincronización entre la pantalla y el servidor: si la ventana de proyección se cierra y se vuelve a abrir, recupera el estado vigente sin ningún mecanismo adicional. El volumen de datos lo permite, dado que con treinta pulsadores el mensaje es de unos pocos kilobytes.

4.4.4 *Software* que se ejecuta en la PC

La aplicación que se ejecuta en la computadora del coordinador se organiza en siete módulos: la configuración, el acceso a la base de datos SQLite, el motor de la partida, el cálculo de puntaje, el cliente de mensajería, el bus de eventos hacia los navegadores y el servidor web. El cliente de mensajería y el servidor web comparten memoria dentro de un único proceso, de modo que el estado de la partida reside en el motor.

La validación de cada voto concentra las reglas del juego:

```
def votar(self, pulsador_id, opcion, seq=None):
    self.visto(pulsador_id)
    with self._lock:
        if seq is not None:
            clave = (pulsador_id, seq)
            if clave in self._seq_vistos:
                return "duplicado"
            self._seq_vistos.append(clave)

        if self.estado != PREGUNTA_ABIERTA:
            return "fuera_de_ventana"

        preg = self._pregunta_actual()
        if opcion not in config.OPCIONES[: len(preg["opciones"])]:
            return "opcion_invalida"
```

```

    caja = self.votos.setdefault(preg["orden"], {})
    if pulsador_id in caja:
        return "ya_voto"          # primera pulsación gana

    caja[pulsador_id] = {
        "pulsador": pulsador_id,
        "opcion": opcion,
        "recibido_en": time.monotonic(),
    }
    self._emitir()
    return None

```

Los cuatro motivos de descarte implementan los requerimientos RF-2 y RF-3. El registro del instante de recepción es el que determina el orden de llegada y, con él, el puntaje. El cierre de la ventana no depende de la interfaz: un hilo independiente verifica el vencimiento del temporizador cada 200 ms, de modo que la pregunta se cierra aunque no haya ninguna pantalla abierta.

El reparto de puntaje de la modalidad competitiva se resuelve en un módulo propio:

```

aciertos = sorted(
    (v for v in votos if v["opcion"] == correcta),
    key=lambda v: v["recibido_en"],
)
asignados = {}
for i, v in enumerate(aciertos, start=1):
    asignados[v["pulsador"]] = puntos(i, puntaje_base)

```

Se ordenan únicamente las respuestas correctas por instante de recepción y se aplica el porcentaje correspondiente a la posición: 100 % para la primera, 90 % para la segunda, 80 % para la tercera y 60 % de la cuarta en adelante. Las respuestas incorrectas no suman, cualquiera sea su orden de llegada.

4.5 Problemas encontrados y soluciones

Entrega de materiales pendiente. Es el factor de mayor impacto sobre el cronograma: condiciona las cuatro tareas en plazo de la Tabla 7 y, de prolongarse, comprometería su cumplimiento al cierre del mes. Su efecto se mitigó mediante el entorno de simulación previsto en el Plan, que permitió desarrollar y validar la aplicación completa sin equipamiento. La planificación de las tareas dependientes del *hardware* se detalla en la sección 4.6.

Deduplicación de votos ante el reinicio de un nodo. La aplicación descarta los reenvíos registrando los pares de identificador y número de secuencia ya procesados. Durante las pruebas se detectó que, al reiniciarse un pulsador, su contador vuelve a comenzar en uno y la aplicación interpreta los nuevos votos como reenvíos, descartándolos en forma

silenciosa durante el resto de la sesión. Con equipamiento real el defecto se manifestaría ante cualquier reinicio por corte de alimentación. La solución propuesta consiste en acotar la ventana de deduplicación a la pregunta en curso en lugar de mantenerla para toda la sesión; se evaluó también incorporar un identificador de arranque al mensaje, alternativa descartada por requerir una modificación del protocolo.

Limitación de la biblioteca MQTT para microcontroladores. Al documentar el protocolo se verificó que PubSubClient, la biblioteca más difundida para ESP8266, solo admite publicación con calidad de servicio 0, mientras que el protocolo definido especifica calidad 1 para los votos [6]. El hallazgo es relevante porque la biblioteca no informa la limitación: la publicación se realiza en calidad 0 sin notificación alguna. Se encuentra pendiente la decisión entre adoptar una biblioteca que admita calidad 1 o aceptar calidad 0 y actualizar el protocolo en consecuencia.

Configuración por omisión del *broker*. Mosquitto, en su versión 2, atiende únicamente conexiones locales y rechaza las anónimas, de modo que ningún pulsador puede conectarse sin una configuración explícita. Se resolvió declarando el puerto de escucha en todas las interfaces y habilitando las conexiones sin credenciales, admisible dado que la red del aula es cerrada y sin salida a internet.[7]

Mensajes de presencia de sesiones anteriores. Los mensajes de presencia se publican como retenidos, por lo que al iniciarse la aplicación el intermediario entrega los de sesiones previas y pulsadores ya ausentes aparecían como presentes. Se resolvió distinguiendo los mensajes retenidos de los latidos reales: los primeros registran al pulsador pero no lo dan por activo hasta recibir un latido efectivo.

Obstáculos previstos en la configuración de la Raspberry Pi. La investigación realizada identificó cuatro cuestiones a resolver antes de la implementación: existen reportes de bloqueo del controlador inalámbrico de la Raspberry Pi 3 en torno a los veinte dispositivos asociados, limitación de *hardware* que se resuelve con un adaptador externo; la versión Bookworm del sistema operativo migró a NetworkManager y ya no utiliza el archivo de configuración anterior, por lo que la interfaz debe excluirse de su gestión; conviene asignar concesiones de dirección de corta duración para que los pulsadores que se reconectan conserven su dirección; y resta medir la ocupación de los canales 1, 6 y 11 —los únicos que no se solapan en la banda de 2,4 GHz— para seleccionar el de mejor calidad. Ninguna de estas cuestiones pudo abordarse por falta de equipamiento.

4.6 Planificación de las partes faltantes

La Tabla 9 detalla la planificación de las tareas pendientes. Las dos primeras son de *software* y no tienen dependencias externas, por lo que se fechan dentro de octubre. Las restantes comienzan con la recepción de los materiales, de modo que sus plazos se expresan en relación con ese hito y no con una fecha de calendario.

Tarea	Dependencia	Plazo previsto
Corrección de la deduplicación de votos	Ninguna	Segunda semana de octubre
Decisión sobre la calidad de servicio y actualización del protocolo	Ninguna	Segunda semana de octubre
<i>Firmware</i> del pulsador	Materiales	Primera y segunda semana desde la recepción
Configuración de la Raspberry Pi como punto de acceso	Materiales	Primera semana desde la recepción
Armado del primer pulsador y prueba de extremo a extremo	Materiales y <i>firmware</i>	Segunda semana desde la recepción
Armado del segundo pulsador y montaje en gabinete	Primer pulsador validado	Tercera semana desde la recepción
Medición de consumo y prueba de autonomía	Pulsadores armados	Tercera semana desde la recepción
Prueba de concurrencia con los dos pulsadores físicos	Pulsadores armados	Tercera semana desde la recepción
Medición de latencia contra el requerimiento RT-1	Pulsadores armados	Cuarta semana desde la recepción
Ensayo completo de una sesión de votación	Todo lo anterior	Cuarta semana desde la recepción

Tabla 9. Planificación cronológica de las tareas faltantes.

El cumplimiento de las cuatro tareas comprometidas para el cierre de octubre depende de que los materiales se reciban dentro de la primera quincena del mes. De producirse una demora mayor, esas tareas se trasladarían al período de noviembre, lo que comprimiría el plazo disponible para las mediciones sobre equipamiento real. En ese escenario se priorizaría el armado de un único pulsador y la demostración de extremo a extremo, dejando la segunda unidad y las mediciones de autonomía para el tramo final de la cursada.

5.- Documentación Relacionada

5.1 Interfaz web construida

Las figuras siguientes corresponden a la interfaz web descrita en la sección 4.4.3, capturadas durante una sesión de prueba con pulsadores simulados. La Figura 3 muestra el editor de encuestas, perteneciente al panel del coordinador.

ClickAula Encuestas Historial broker conectado 10 pulsadores en línea Abrir proyección ↗

Nueva encuesta

Nombre * Repaso de sistemas distribuidos Modalidad Competencia por pu Segundos por pregunta 30 Puntaje base 500

Preguntas

1. Escribí la pregunta ↑ ↓ [icon] [x]

A Opción obligatoria ☐ correcta

B Opción obligatoria ☐ correcta

C Opción (podés dejarla vacía) ☐ correcta

D Opción (podés dejarla vacía) ☐ correcta

Segundos (vacío = el de la encuesta) Sin respuesta correcta

+ Agregar pregunta

Cancelar Crear encuesta

Figura 3. Editor de encuestas: alta de preguntas con sus opciones y la respuesta correcta.

La Figura 4 muestra el panel del coordinador con una pregunta abierta. Se observan el temporizador, el contador de votos recibidos sobre el total esperado y la grilla de pulsadores, donde se destacan los que ya respondieron.

ClickAula Encuestas Historial broker conectado 10 pulsadores en línea Abrir proyección ↗

PREGUNTA 1 DE 5

¿Qué protocolo usa ClickAula para los votos?

Tiempo Votos 18 9/10 Cerrar ahora

A HTTP

B MQTT

C FTP

D SMTP

QUIÉN YA VOTÓ

01 voto 02 voto 03 voto 04 voto 05 voto 06 voto 07 voto 08 en línea 09 voto 10 voto

Figura 4. Panel del coordinador durante una pregunta abierta, con el temporizador, el contador de votos y la grilla de pulsadores.

La Figura 5 corresponde a la vista de proyección durante esa misma pregunta. Las opciones se presentan con el color del botón físico correspondiente y con un símbolo distinto cada una, de modo que resulten distinguibles para quienes no perciben diferencias de color.



Figura 5. Vista de proyección durante una pregunta abierta.

La Figura 6 muestra los resultados de la pregunta una vez cerrada la votación, con la distribución por opción, la identificación de la respuesta correcta, los participantes que acertaron con mayor rapidez y la tabla de posiciones acumulada.

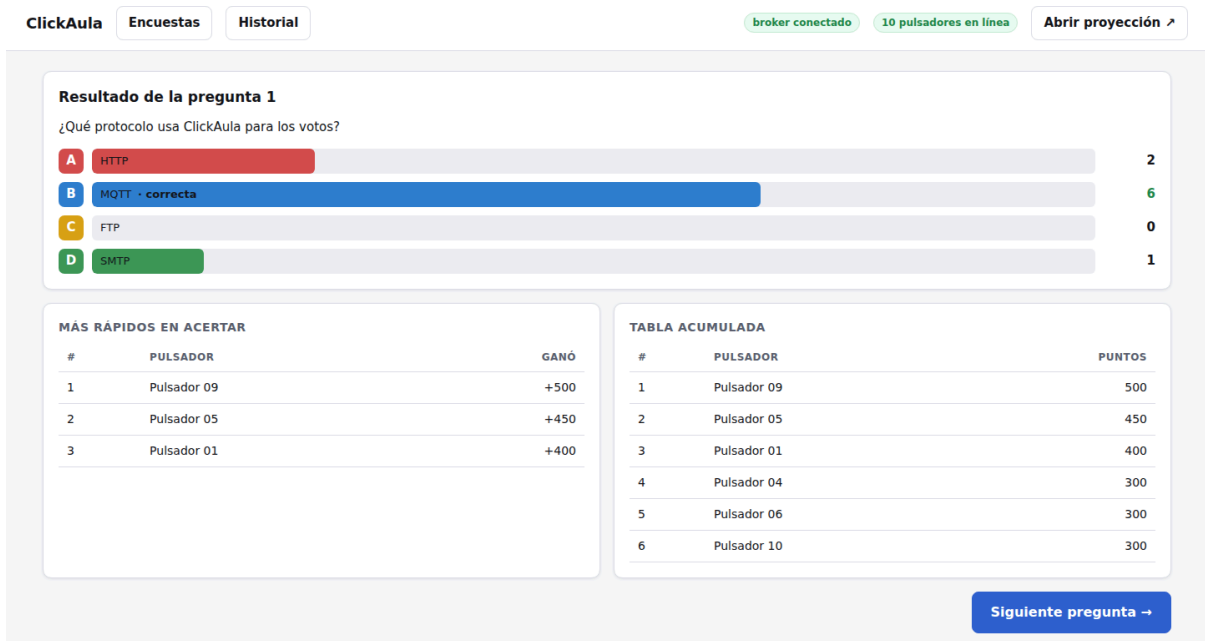


Figura 6. Resultados de una pregunta en el panel del coordinador, con la distribución por opción y la tabla acumulada.

La Figura 7 presenta la pantalla final de proyección, con el podio y las preguntas de mayor y menor porcentaje de acierto.



Figura 7. Pantalla final de proyección con el podio y las preguntas de mayor y menor porcentaje de acierto.

5.2 Partes físicas construidas

No se incluyen fotografías de partes físicas construidas: el armado comienza con la recepción de los materiales, pendiente a la fecha según se detalla en la sección 2.2. El esquema de conexiones ya definido se presenta en la Figura 2.

5.3 Enlaces

Video de demostración. [completar con el enlace al video]. El video registra una sesión completa de votación en sus dos modalidades, ejecutada sobre el entorno de simulación.

Bitácora del proyecto. [completar con el enlace a la bitácora].

Repositorio de código fuente. [completar con el enlace al repositorio].

Se otorgó acceso de edición al docente a cargo del grupo tanto a la bitácora como al repositorio, según lo solicitado. El repositorio incluye la aplicación completa, el simulador, el entorno de ejecución y la documentación del protocolo de mensajería destinada al desarrollo del *firmware*.

6.- Bibliografía

[1] OASIS. *MQTT Version 5.0. OASIS Standard*. 2019. Disponible en:

<https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (consultado el 3 de octubre de 2026).

[2] Espressif Systems. *ESP8266EX Datasheet*. Versión 7.0, 2023. Disponible en:

https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf (consultado el 3 de octubre de 2026).

[3] Raspberry Pi Ltd. *Raspberry Pi Documentation — Power Supply*. Disponible en:

<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#power-supply> (consultado el 3 de octubre de 2026).

[4] Pallets Projects. *Flask Documentation*. Disponible en: <https://flask.palletsprojects.com/> (consultado el 3 de octubre de 2026).

[5] Eclipse Foundation. *Eclipse Paho MQTT Python Client*. Disponible en:

<https://eclipse.dev/paho/index.php?page=clients/python/index.php> (consultado el 3 de octubre de 2026).

[6] O'Leary, N. *PubSubClient — API Documentation*. Disponible en:

<https://pubsubclient.knolleary.net/api> (consultado el 3 de octubre de 2026).

[7] Eclipse Foundation. *Eclipse Mosquitto — mosquitto.conf Manual Page*. Disponible en:

<https://mosquitto.org/man/mosquitto-conf-5.html> (consultado el 3 de octubre de 2026).

Nota: la referencia [7] documenta la configuración del broker. Los reportes sobre el límite de dispositivos asociados en la Raspberry Pi 3 citados en la sección 4.5 provienen de los foros oficiales de la plataforma y deberán incorporarse con su cita precisa una vez verificados sobre el equipamiento propio.